

Phase#4: Core Functionality Prototype

UniConnect

Github : [UniConnect](#)

Zewail City of Science, Technology and Innovation

CSAI 203- Fall 2025

contact: s-nour.ghoniem@zewailcity.edu.eg

Members

NAME	Id
Nour Hosam	202401560
Ganna Allah	202401020
Shaza Kazem	202400948
Shehab Osama	202402196
Malak Mohamed	202401538



1. Introduction

The objective of Phase 4 was to deliver a running prototype with core functionalities implemented. This report summarizes the features completed, the architectural patterns applied, and the features remaining for the final delivery.

2. Functional Requirements Status & Completion Percentage

We have completed **9 out of 14** functional requirements defined in our SRS, achieving a **64% Completion Rate** for this phase.

Feature Name	Status	Implementation Details / Notes
User Registration & Login	✅ Completed	Users can register and log in with role-based access (Admin/Student).
Search & Filter	✅ Completed	Implemented search functionality for Clubs and Offices by name.
View Club Details	✅ Completed	Students can view full details of university clubs and staff members.
View Office Details	✅ Completed	Students can view details and staff for university offices.
Messaging & Contact	✅ Completed	Users can send messages.
Notification & Inbox	✅ Completed	Users have an inbox to receive messages and notifications.
Add New Club/Office	✅ Completed	Admins can create new entities via the Dashboard.
Edit or Delete Club/Office	✅ Completed	Admins can update or remove entities entirely.
Assign Hierarchical Roles	✅ Completed	Admins can assign roles (Head, Vice, Member) to users.
Join Club Request	⌚ Pending	Logic for students to send join requests is scheduled for Phase 5.
Approve/Reject Requests	⌚ Pending	Logic for leaders to manage requests is scheduled for Phase 5.
Profile Management	⌚ Pending	User ability to edit their own profile is scheduled for Phase 5.
Post Announcements/Updates	⌚ Pending	System for admins to post news is scheduled for Phase 5.
Reports & Analytics	⌚ Pending	Dashboard for viewing system statistics is scheduled for Phase 5.

3. Architecture & Design Consistency

The implementation follows the **Model-View-Controller (MVC)** pattern as defined in our Design Document.

- **Model:** Data is managed via Repositories using file-based storage (CSV).
- **View:** HTML templates & CSS.
- **Controller:** Logic is separated into Flask Blueprints.

4. Technical Achievements & Bonus Features

We have successfully implemented **4 Bonus Features** (Totaling 3 Bonus Grades) to improve code quality, maintainability, and user experience, as outlined in the project guidelines :

1. **Modularization (Blueprints) [Bonus 1 Grade]:** We split the large Flask application into distinct modules using Flask Blueprints.
2. **Repository Pattern [Bonus 1 Grade]:** We isolated all data access logic into a dedicated repositories. The Controllers never interact with CSV files directly; they request objects from the Repository.
3. **Singleton Pattern [Bonus 0.5 Grade]:** We implemented the **Singleton Pattern** for our FileManager class, to handle all CSV connections .
4. **Use of CSS for Improved UI [Bonus 0.5 Grade]:** We implemented a custom style.css file to ensure a consistent theme, clean layout, and responsive design across all pages, improving the overall user experience.

5. Next Steps (Phase 5 Plan)

To reach 100% functionality for the Final Delivery, the team will focus on implementing the 5 pending features listed above, followed by unit testing.